

Linux Assignment-1

Completed

Linux Assignment-2

1. Search for a specific word "apple" in a file and count it.

```
vipultalele@vipul04:~$ grep -o "apple" filename.txt | wc -l
grep: filename.txt: No such file or directory
0
vipultalele@vipul04:~$
```

```
vipultalele@vipul04:~$ grep -o "apple" fruits.txt | wc -l
2
vipultalele@vipul04:~$ |
```

2. Show current users logged in.

```
vipultalele@vipul04:~$ who
vipultalele pts/0      2026-04-07 09:38 (10.0.2.2)
vipultalele@vipul04:~$ |
```

3. Show total number of .txt files in current directory.

```
vipultalele@vipul04:~$ ls *.txt 2>/dev/null | wc -l
4
vipultalele@vipul04:~$ |
```

4. Show list of all system users from /etc/passwd.

```

vipultalele@vipul04:~$ cut -d: -f1 /etc/passwd
root
daemon
bin
sys
sync
games
man
lp
mail
news
uucp
proxy
www-data
backup
list
irc
_apt
nobody
systemd-network
systemd-timesync
dhcpcd
messagebus
systemd-resolve
pollinate
polkitd
syslog
uidd
tcpdump
tss
landscape
fwupd-refresh
usbmux
vipultalele
sshd
nov2025
vipultalele@vipul04:~$ |

```

5. Find how many times a word and each match should be on newline.

```

vipultalele@vipul04:~$ grep -o "word" filename.txt | wc -l
grep: filename.txt: No such file or directory
0

```

6. Show running processes and search for "root"

```

vipultalele@vipul04:~/nov-2025$ ps aux | grep root
root      1  0.1  0.3 22044 13204 ?        Ss   09:37   0:00 /sbin/init splash noprompt noshell automatic-ubiquity
root      2  0.0  0.0      0   0 ?        S    09:37   0:00 [kthreadd]
root      3  0.0  0.0      0   0 ?        S    09:37   0:00 [pool_workqueue_release]
root      4  0.0  0.0      0   0 ?        I<   09:37   0:00 [kworker/R-rcu_g]
root      5  0.0  0.0      0   0 ?        I<   09:37   0:00 [kworker/R-rcu_p]
root      6  0.0  0.0      0   0 ?        I<   09:37   0:00 [kworker/R-slub_]
root      7  0.0  0.0      0   0 ?        I<   09:37   0:00 [kworker/R-netns]
root     11  0.0  0.0      0   0 ?        I    09:37   0:00 [kworker/u4:0-ipv6_addrconf]

```

7. Show only .log files and sort them alphabetically.

```

vipultalele@vipul04:~$ ls *.log 2>/dev/null | sort
vipultalele@vipul04:~$ |

```

```

vipultalele@vipul04:/var/log$ ls *.log
alternatives.log  apport.log  auth.log  boot.log  bootstrap.log  cloud-init.log  cloud-init-output.log  dpkg.log  kern.log  vboxpostinstall.log
vipultalele@vipul04:/var/log$ |

```

8. See disk space and filter "drivers".

```

vipultalele@vipul04:~$ df -h | grep /dev/sda2
/dev/sda2      25G   4.1G   20G   18% /
vipultalele@vipul04:~$ |

```

9. Sort names alphabetically and remove duplicates.

```

vipultalele@vipul04:~$ sort fruits.txt | uniq
Apple
banana
Pineapple
watermelone
vipultalele@vipul04:~$ |

```

10. Find all .log files modified in last 1 day.

```

vipultalele@vipul04:/var/log$ sudo su
[sudo] password for vipultalele:
root@vipul04:/var/log# find . -name "*.log" -mtime -1
./kern.log
./unattended-upgrades/unattended-upgrades-dpkg.log
./unattended-upgrades/unattended-upgrades.log
./auth.log
./dpkg.log
./boot.log
./apt/term.log
./apt/history.log
root@vipul04:/var/log# |

```

11. Delete .tmp files older than 7 days.

```

vipultalele@vipul04:/tmp$ sudo su
root@vipul04:/tmp# find . -name "*.tmp" -mtime +7 -delete
root@vipul04:/tmp# |

```

12. Count how many .java files exist in a project.

```

vipultalele@vipul04:~$ sudo su
root@vipul04:/home/vipultalele# find . -name "*.java" | wc -l
0
root@vipul04:/home/vipultalele# |

```

- Count how many times the log was accessed.

```

vipultalele@vipul04:~$ vim access.log
vipultalele@vipul04:~$ wc -l access.log
8 access.log

```

- Show only lines where status code is 404

```

vipultalele@vipul04:~$ grep ' 404 ' access.log
192.168.0.2 - - [03/Aug/2025:10:01:27 +0000] "GET /contact HTTP/1.1" 404 256
vipultalele@vipul04:~$ |

```

- List all unique IPs that accessed the server. Please note, IP means example, 192.168.0.2.

```

vipultalele@vipul04:~$ awk '{print $1}' access.log | sort -u
192.168.0.2
192.168.0.3
192.168.0.4
vipultalele@vipul04:~$ |

```

- Count how many requests were POST

```

vipultalele@vipul04:~$ grep '"POST' access.log | wc -l
0
vipultalele@vipul04:~$ |

```

- List all requested URLs

```

vipultalele@vipul04:~$ awk '{print $7}' access.log
/index.html
/login
/about
/index.html
/contact
/profile
/products
/cart
vipultalele@vipul04:~$ |

```

- Find which IPs triggered 404 errors

```

vipultalele@vipul04:~$ grep ' 404 ' access.log | awk '{print $1}' | sort -u
192.168.0.2
vipultalele@vipul04:~$ |

```

Linux Assignment-3

- Write a script that creates a new directory, then creates three empty files inside it, and finally lists the contents of the directory.

```

vipultalele@vipul04:~/assignment$ vim Q1.sh
vipultalele@vipul04:~/assignment$ chmod +x Q1.sh
vipultalele@vipul04:~/assignment$ bash Q1.sh
total 0
-rw-rw-r-- 1 vipultalele vipultalele 0 Apr  8 10:42 file1.txt
-rw-rw-r-- 1 vipultalele vipultalele 0 Apr  8 10:42 file2.txt
-rw-rw-r-- 1 vipultalele vipultalele 0 Apr  8 10:42 file3.txt
vipultalele@vipul04:~/assignment$ |

```

```

#!/bin/bash

# Create a new directory
mkdir my_directory

# Navigate into the directory
cd my_directory

# Create three empty files
touch file1.txt file2.txt file3.txt

# List the contents of the directory
ls -l

```

2. Create a script that prompts the user for a filename, then checks if the file exists and whether it is a regular file or a directory.

```
#!/bin/bash

# Prompt user for filename
echo "Enter a filename or directory:"
read name

# Check if it exists
if [ -e "$name" ]; then
    echo "It exists."

    # Check if it's a regular file
    if [ -f "$name" ]; then
        echo "It is a regular file."

    # Check if it's a directory
    elif [ -d "$name" ]; then
        echo "It is a directory."

    else
        echo "It exists but is neither a regular file nor a directory."
    fi
fi
else
    echo "It does not exist."
fi

vipultalele@vipul04:~/assignment$ bash Q2.sh
Enter a filename or directory:
Q1.sh
It exists.
It is a regular file.
vipultalele@vipul04:~/assignment$ |
```

3. Write a script to copy a file from a source path to a destination path provided as arguments. Handle cases where source or destination are invalid.

```
#!/bin/bash

# Check if exactly 2 arguments are passed
if [ "$#" -ne 2 ]; then
    echo "Usage: $0 <source_file> <destination_path>"
    exit 1
fi

source_file="$1"
destination_path="$2"

# Check if source exists
if [ ! -e "$source_file" ]; then
    echo "Error: Source file does not exist."
    exit 1
fi

# Check if source is a regular file
if [ ! -f "$source_file" ]; then
    echo "Error: Source is not a regular file."
    exit 1
fi

# Check if destination directory exists
if [ -d "$destination_path" ]; then
    # Copy into directory
    cp "$source_file" "$destination_path"
elif [ -e "$destination_path" ]; then
    # Destination exists but not a directory (overwrite file)
    cp "$source_file" "$destination_path"
else
    echo "Error: Destination path is invalid."
    exit 1
fi

# Check if copy was successful
if [ "$?" -eq 0 ]; then
```

```
# Check if copy was successful
if [ "$?" -eq 0 ]; then
    echo "File copied successfully."
else
    echo "Error occurred during copy."
fi
```

4. Develop a script that displays the current date, time, and your username.

```
#!/bin/bash

# Display current date
echo "Current Date: $(date +%Y-%m-%d)"

# Display current time
echo "Current Time: $(date +%H:%M:%S)"

# Display username
echo "Username: $(whoami)"
```

```
vipultalele@vipul04:~/assignment$ bash Q4.sh
Current Date: 2026-04-08
Current Time: 10:55:44
Username: vipultalele
vipultalele@vipul04:~/assignment$ |
```

5. Create a script that checks if a given file has read, write, and execute permissions for the owner.

```
#!/bin/bash

# Check if argument is provided
if [ "$#" -ne 1 ]; then
    echo "Usage: $0 <filename>"
    exit 1
fi

file="$1"

# Check if file exists
if [ ! -e "$file" ]; then
    echo "Error: File does not exist."
    exit 1
fi

# Check owner permissions
echo "Checking permissions for: $file"

if [ -r "$file" ]; then
    echo "Owner has READ permission."
else
    echo "Owner does NOT have READ permission."
fi

if [ -w "$file" ]; then
    echo "Owner has WRITE permission."
else
    echo "Owner does NOT have WRITE permission."
fi

if [ -x "$file" ]; then
    echo "Owner has EXECUTE permission."
else
    echo "Owner does NOT have EXECUTE permission."
fi

~
```

6. Develop a script that compares two strings provided as arguments and indicates if they are equal or different.

```
#!/bin/bash

# Check if exactly 2 arguments are passed
if [ "$#" -ne 2 ]; then
    echo "Usage: $0 <string1> <string2>"
    exit 1
fi

str1="$1"
str2="$2"

# Compare strings
if [ "$str1" = "$str2" ]; then
    echo "Strings are equal."
else
    echo "Strings are different."
fi
```

7. Write a script that lists all currently running processes and their PIDs.

```
#!/bin/bash

echo "List of running processes and their PIDs:"
echo "-----"

# List all processes with PID and command
ps -e -o pid,comm
```

```
vipultalele@vipul04:~/assignment$ bash Q7.sh
List of running processes and their PIDs:
-----
  PID COMMAND
    1 systemd
    2 kthreadd
    3 pool_workqueue_release
```


8. Create a script that takes a process ID as an argument and attempts to terminate that process.

```
#!/bin/bash

# Check if PID is provided
if [ "$#" -ne 1 ]; then
    echo "Usage: $0 <PID>"
    exit 1
fi

pid="$1"

# Check if PID is a number
if ! [[ "$pid" =~ ^[0-9]+$ ]]; then
    echo "Error: PID must be a number."
    exit 1
fi

# Check if process exists
if ps -p "$pid" > /dev/null 2>&1; then
    echo "Process $pid found. Attempting to terminate..."

    # Try graceful termination
    kill "$pid"

    sleep 2

    # Check if still running
    if ps -p "$pid" > /dev/null 2>&1; then
        echo "Process still running. Forcing termination..."
        kill -9 "$pid"
    fi

    # Final check
    if ps -p "$pid" > /dev/null 2>&1; then
        echo "Failed to terminate process $pid."
    else
        echo "Process $pid terminated successfully."
    fi
fi
```

9. Develop a script that displays disk usage information for all mounted filesystems.

```
#!/bin/bash

echo "Disk Usage Information:"
echo "-----"

# Display disk usage in human-readable format
df -h
```

```
vipultalele@vipul04:~/assignment$ bash Q9.sh
Disk Usage Information:
-----
Filesystem      Size  Used Avail Use% Mounted on
tmpfs           392M  1.1M  391M   1% /run
/dev/sda2       25G   4.1G   20G  18% /
tmpfs           2.0G    0  2.0G   0% /dev/shm
tmpfs           5.0M    0  5.0M   0% /run/lock
tmpfs           392M   12K  392M   1% /run/user/1000
vipultalele@vipul04:~/assignment$
```

10. Write a script that counts the number of words, lines, and characters in a given text file.

```
#!/bin/bash

# Check if filename is provided
if [ "$#" -ne 1 ]; then
    echo "Usage: $0 <filename>"
    exit 1
fi

file="$1"

# Check if file exists
if [ ! -f "$file" ]; then
    echo "Error: File does not exist or is not a regular file."
    exit 1
fi

# Count lines, words, and characters
lines=$(wc -l < "$file")
words=$(wc -w < "$file")
chars=$(wc -c < "$file")

# Display results
echo "File: $file"
echo "Lines: $lines"
echo "Words: $words"
echo "Characters: $chars"
```

11. Create a script that searches for a specific string within a file or directory recursively

```
vipultalele@vipul04: ~/assign X + v
#!/bin/bash

# Check arguments
if [ "$#" -ne 2 ]; then
    echo "Usage: $0 <search_string> <file_or_directory>"
    exit 1
fi

search="$1"
path="$2"

# Check if path exists
if [ ! -e "$path" ]; then
    echo "Error: File or directory does not exist."
    exit 1
fi

# If it's a file
if [ -f "$path" ]; then
    echo "Searching in file: $path"
    grep -n "$search" "$path"
fi

# If it's a directory
elif [ -d "$path" ]; then
    echo "Searching recursively in directory: $path"
    grep -rn "$search" "$path"
fi

else
    echo "Error: Not a valid file or directory."
fi
```

12. Develop a script that backs up a specific directory to a compressed archive, including a timestamp in the filename.

```
vipultalele@vipul04: ~/assign  ×  +  v

#!/bin/bash

# Check if directory argument is provided
if [ "$#" -ne 1 ]; then
    echo "Usage: $0 <directory_to_backup>"
    exit 1
fi

dir="$1"

# Check if directory exists
if [ ! -d "$dir" ]; then
    echo "Error: Directory does not exist."
    exit 1
fi

# Get current timestamp
timestamp=$(date +"%Y%m%d_%H%M%S")

# Extract directory name
dirname=$(basename "$dir")

# Create backup filename
backup_file="${dirname}_backup_${timestamp}.tar.gz"

# Create compressed archive
tar -czf "$backup_file" "$dir"

# Check if backup was successful
if [ "$?" -eq 0 ]; then
    echo "Backup successful: $backup_file"
else
    echo "Error: Backup failed."
fi
```

13. Write a script that schedules a task to run at a specific time using cron.

```
vipultalele@vipul04: ~/assign × + v
#!/bin/bash

# Ask user for schedule details
echo "Enter minute (0-59):"
read min

echo "Enter hour (0-23):"
read hour

echo "Enter command to schedule:"
read command

# Create cron job entry
cron_job="$min $hour * * * $command"

# Add to crontab
(crontab -l 2>/dev/null; echo "$cron_job") | crontab -

echo "Cron job scheduled: $cron_job"
~
```

14. Write a script to replace all occurrences of one word with another throughout data.txt.

```
vipultalele@vipul04: ~/assign × + v
#!/bin/bash

# Check if exactly 2 arguments are passed
if [ "$#" -ne 2 ]; then
    echo "Usage: $0 <old_word> <new_word>"
    exit 1
fi

old_word="$1"
new_word="$2"
file="data.txt"

# Check if file exists
if [ ! -f "$file" ]; then
    echo "Error: $file does not exist."
    exit 1
fi

# Replace all occurrences
sed -i "s/$old_word/$new_word/g" "$file"

echo "All occurrences of '$old_word' replaced with '$new_word' in $file."
~
```

15. Print only the lines from data.txt where the second field is a number greater than 10. Use awk command.

```
vipultalele@vipul04: ~/assign × + v
#!/bin/bash

file="data.txt"

# Check if file exists
if [ ! -f "$file" ]; then
    echo "Error: $file does not exist."
    exit 1
fi

# Use awk to filter lines
awk '$2 ~ /^[0-9]+$/ && $2 > 10' "$file"
```

16. Print lines from data.txt that contain a specific pattern, and then print a custom message for each matched line.

```
vipultalele@vipul04: ~/assign × + v
#!/bin/bash

file="data.txt"

# Check if file exists
if [ ! -f "$file" ]; then
    echo "Error: $file does not exist."
    exit 1
fi

# Ask user for pattern
echo "Enter pattern to search:"
read pattern

# Search and print custom message
grep "$pattern" "$file" | while read line
do
    echo "Matched line: $line"
done
```